

# **Requirements Analysis and Specification Document (RASD)**

**Project: City-Level Heat Monitoring using Meteostat**

**Course: Software Engineering for Geoinformatics – Politecnico di Milano**

**Academic Year: 2025/2026**

| <b>Date</b>       | <b>Authors</b>                                                                                    |
|-------------------|---------------------------------------------------------------------------------------------------|
| <b>24/05/2026</b> | <b>Mohammadmahdi Najafi, Forough Mazidi,<br/>John Junior Frimpong Donkor and Chao Tung<br/>Wu</b> |

---

## **Table of Contents**

1. Introduction
  - 1.1 Purpose
  - 1.2 Context and Motivations
  - 1.3 Definitions, Acronyms, and Abbreviations
  - 1.4 Scope and Limitations
2. Requirement Analysis
  - 2.1 System Boundaries
  - 2.2 External Actors
  - 2.3 Relevant Phenomena
3. Users
4. Use Cases
  - 4.1 User Stories
  - 4.2 Use Case Diagram
  - 4.3 Detailed Use Case Descriptions
5. Class Diagram
6. Functional Requirements
7. Non-Functional Requirements
8. Constraints and Dependencies
  - 8.1 Technical Constraints
  - 8.2 Data Constraints
  - 8.3 Legal and Organisational Constraints
  - 8.4 External Tool Dependencies
9. Future Considerations

## 10. Appendices and References

### 10.1 External Data Sources

### 10.2 Tools, Libraries and Frameworks<sup>1</sup>

### 10.3 Internal Artefacts and Diagrams

### 10.4 Acronyms and Abbreviations

---

## 1 Introduction

### 1.1 Purpose

This Requirements Analysis and Specification Document (RASD) defines the functional and non-functional requirements of the project “City-Level Heat Monitoring using Meteostat”, developed within the Software Engineering for Geoinformatics course at Politecnico di Milano during the academic year 2025/2026.

The goal of the project is to design and develop a client-server application capable of retrieving, storing, processing, and visualizing weather-station observations from Meteostat to support urban heat monitoring and meteorological analysis.

The system will allow users to:

- Query historical hourly meteorological observations.
  - Explore station metadata and weather conditions.
  - Visualize weather data using interactive maps and charts.
  - Analyze temperature trends and heat-related indicators.
  - Export processed datasets for external analysis.
- 

### 1.2 Context and Motivations

Climate change is increasing the frequency and intensity of heatwaves and extreme temperature events in urban environments. Cities are especially vulnerable due to the Urban Heat Island effect, high population density, and built-up surfaces that retain heat.

Public authorities, urban planners, environmental agencies, and researchers increasingly require accessible meteorological information to:

- Monitor urban heat conditions.
- Identify critical heat periods.
- Support climate adaptation strategies.
- Improve public awareness regarding heat-related risks.

Although Meteostat provides detailed weather-station observations through APIs, the raw datasets are difficult to process and interpret for non-technical users.

The proposed application addresses this issue by offering:

- A centralized platform for weather data access.
- Interactive geographic visualization.
- Temporal and statistical analysis tools.
- A modular and extensible architecture.

---

### 1.3 Definitions, Acronyms, and Abbreviations

| Term               | Definition                                                |
|--------------------|-----------------------------------------------------------|
| RASD               | Requirements Analysis and Specification Document          |
| API                | Application Programming Interface                         |
| Meteostat          | Meteorological platform providing historical weather data |
| RapidAPI           | API gateway used to access Meteostat services             |
| Flask              | Python web framework used for backend development         |
| PostgreSQL/PostGIS | Relational database with geospatial support               |
| JSON               | JavaScript Object Notation                                |
| Jupyter Notebook   | Interactive dashboard environment                         |
| UI                 | User Interface                                            |
| UX                 | User Experience                                           |
| Station            | Weather-monitoring location                               |
| Observation        | Hourly meteorological measurement                         |
| Heatwave           | Prolonged period of high temperature                      |
| Urban Heat Island  | Urban area warmer than surrounding rural regions          |
| Dashboard          | Interactive visualization client                          |

---

## 1.4 Scope and Limitations

### In Scope

- Historical hourly weather observations for at least two cities.
- Retrieval of station metadata from Meteostat.
- Interactive maps and charts.
- Filtering by city, station, date range, and weather variable.
- Export of weather datasets.
- Storage of observations in PostgreSQL/PostGIS.
- Backend preprocessing and cleaning of data.

### Out of Scope

- Weather forecasting.
- Machine-learning prediction models.
- Mobile-native applications.
- Satellite imagery integration.
- Global-scale monitoring.

### Limitations

- RapidAPI free subscription is limited to 500 requests/month.
- Data completeness depends on Meteostat availability.
- System optimized for desktop environments.
- Continuous ingestion may not be fully automated.

---

## 2 Requirement Analysis

### 2.1 System Boundaries

The system includes:

- A PostgreSQL/PostGIS database for storing station metadata and weather observations.
- A Flask backend server exposing REST APIs.
- A Jupyter Notebook dashboard for visualization and analysis.
- Integration with Meteostat through RapidAPI.

The system does not include:

- Weather prediction services.
- Mobile-native interfaces.
- Third-party climate analytics systems.

---

## 2.2 External Actors

| Actor                | Description                                        |
|----------------------|----------------------------------------------------|
| Unauthenticated User | Visitor exploring public dashboard features        |
| Authenticated User   | Registered user accessing advanced functionalities |
| Meteostat API        | External provider of weather observations          |
| RapidAPI             | External gateway for Meteostat access              |

---

## 2.3 Relevant Phenomena

The system models and analyzes the following real-world phenomena:

- Hourly temperature variations.
  - Urban heat accumulation.
  - Warm-night occurrences.
  - Heatwave duration.
  - Spatial distribution of weather stations.
  - Temporal meteorological trends.
- 

## 3 Users

| User Type            | Description     | Main Capabilities                                 |
|----------------------|-----------------|---------------------------------------------------|
| Unauthenticated User | Public visitor  | Explore dashboard previews and maps               |
| Authenticated User   | Registered user | Advanced querying, exporting, and personalization |

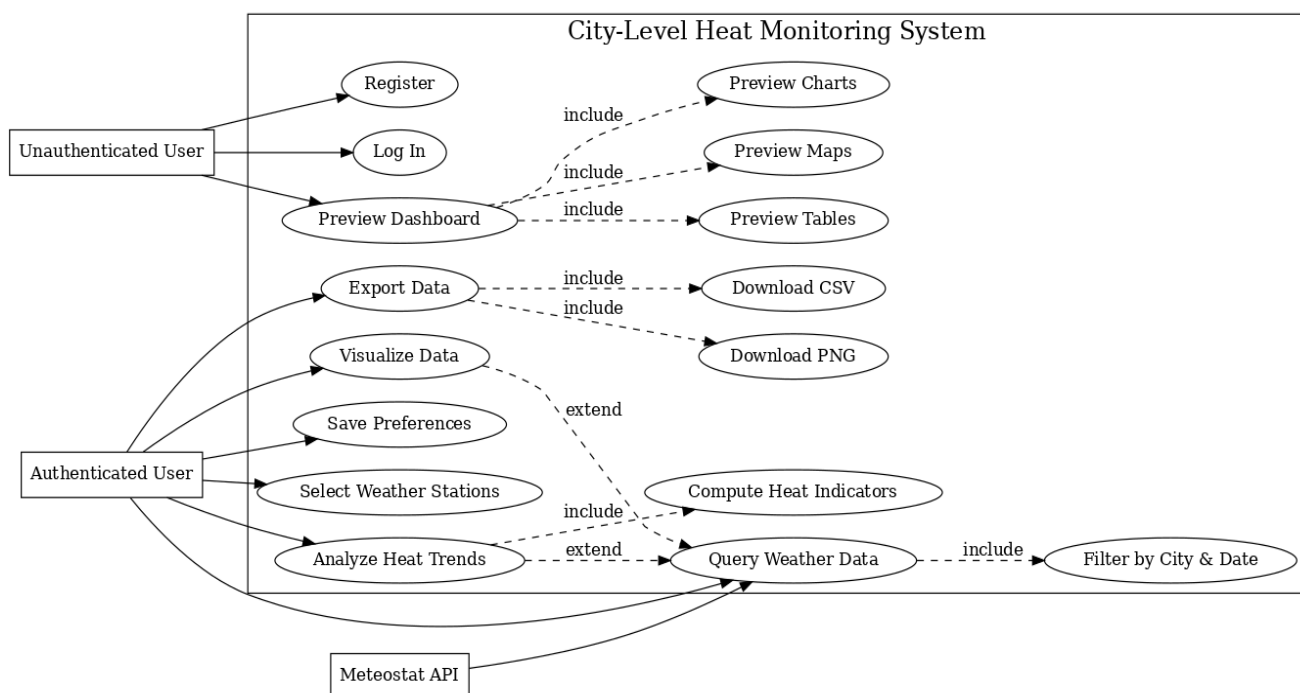
---

## 4 Use Cases

### 4.1 User Stories

| ID   | User                 | Action                           | Benefit                               |
|------|----------------------|----------------------------------|---------------------------------------|
| US-1 | Unauthenticated User | Explore heat maps                | Understand urban temperature patterns |
| US-2 | Unauthenticated User | Preview charts and tables        | Evaluate platform capabilities        |
| US-3 | Authenticated User   | Query weather data               | Analyze meteorological conditions     |
| US-4 | Authenticated User   | Filter stations by city and date | Focus analysis                        |
| US-5 | Authenticated User   | Export datasets                  | Perform external processing           |
| US-6 | Authenticated User   | Save dashboard preferences       | Reuse preferred configurations        |
| US-7 | Authenticated User   | Compare stations                 | Study spatial heat variability        |

### 4.2 Use Case Diagram



The use case diagram includes:

- Actors
- Authentication operations
- Querying operations
- Visualization functionalities
- Export functionalities
- Heat analysis operations
- Relationships between use cases

---

### 4.3 Detailed Use Case Descriptions

#### UC-1 Query Weather Data

| Field            | Description                                                                                                                 |
|------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-1                                                                                                                        |
| Name             | Query Weather Data                                                                                                          |
| Primary Actor    | Authenticated User                                                                                                          |
| Preconditions    | User is logged in                                                                                                           |
| Trigger          | User selects city, station, and date range                                                                                  |
| Main Flow        | 1. User selects filters 2. Backend validates parameters 3. Database query executed 4. Results returned 5. Dashboard updated |
| Alternative Flow | No observations available                                                                                                   |
| Postconditions   | Weather observations displayed                                                                                              |

---

#### UC-2 Visualize Heat Trends

| Field       | Description |
|-------------|-------------|
| Use Case ID | UC-2        |

| Field            | Description                                                                   |
|------------------|-------------------------------------------------------------------------------|
| Name             | Visualize Heat Trends                                                         |
| Primary Actor    | Authenticated User                                                            |
| Preconditions    | Weather data available                                                        |
| Trigger          | User selects visualization mode                                               |
| Main Flow        | 1. User selects charts/maps 2. System computes indicators 3. Graphs displayed |
| Alternative Flow | Invalid visualization parameters                                              |
| Postconditions   | Interactive charts shown                                                      |

---

### UC-3 Export Meteorological Data

| Field            | Description                                                               |
|------------------|---------------------------------------------------------------------------|
| Use Case ID      | UC-3                                                                      |
| Name             | Export Meteorological Data                                                |
| Primary Actor    | Authenticated User                                                        |
| Preconditions    | Query results available                                                   |
| Trigger          | User clicks export button                                                 |
| Main Flow        | 1. User selects export format 2. System generates file 3. Download starts |
| Alternative Flow | Export generation error                                                   |
| Postconditions   | File downloaded                                                           |

---

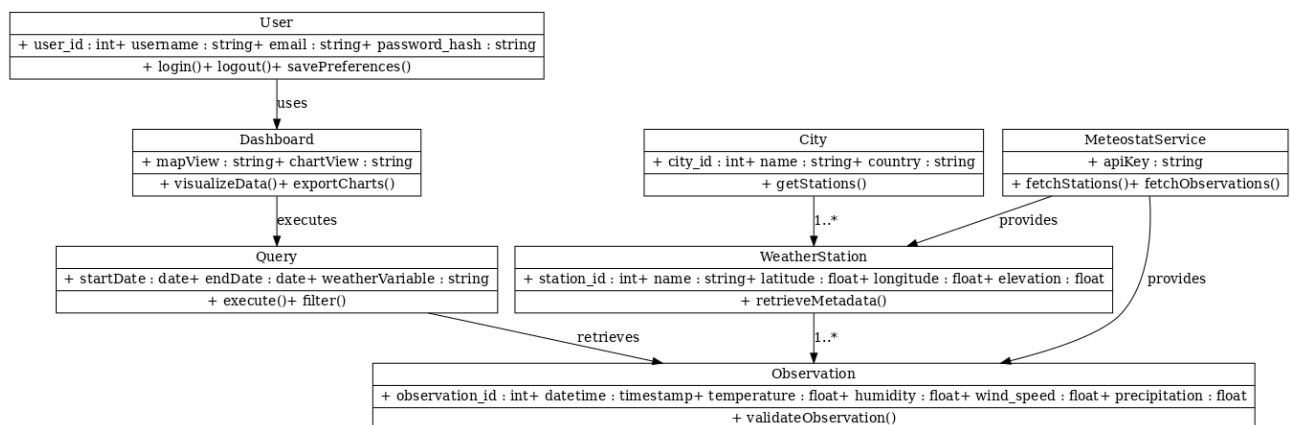
### UC-4 Save Dashboard Preferences

| Field       | Description                |
|-------------|----------------------------|
| Use Case ID | UC-4                       |
| Name        | Save Dashboard Preferences |



| Field            | Description                                                                  |
|------------------|------------------------------------------------------------------------------|
| Primary Actor    | Authenticated User                                                           |
| Preconditions    | User authenticated                                                           |
| Trigger          | User saves filters or map view                                               |
| Main Flow        | 1. User configures dashboard 2. Preferences stored 3. Confirmation displayed |
| Alternative Flow | Database storage failure                                                     |
| Postconditions   | Preferences available for future sessions                                    |

## 5 Class Diagram



Main classes included:

- User
- City
- WeatherStation
- Observation
- Query
- Dashboard
- MeteostatService

Relationships:

- A city contains multiple weather stations.

- A weather station contains multiple observations.
  - Dashboard executes queries.
  - MeteostatService retrieves stations and observations.
- 

## **6 Functional Requirements**

### **F1 – Authentication and User Management**

#### **F1.1 Registration**

The system shall allow users to create accounts.

#### **F1.2 Login**

The system shall authenticate users.

#### **F1.3 Logout**

The system shall invalidate user sessions.

---

### **F2 – Weather Data Querying**

#### **F2.1 City Selection**

Users shall select one or more cities.

#### **F2.2 Station Selection**

Users shall select weather stations.

#### **F2.3 Date Filtering**

Users shall filter observations by time interval.

#### **F2.4 Weather Variable Selection**

Users shall select variables including:

- Temperature
- Humidity
- Wind speed
- Precipitation

#### **F2.5 Query Execution**

The system shall retrieve matching observations.

---

## **F3 – Visualization**

### **F3.1 Interactive Maps**

The system shall display weather stations on maps.

### **F3.2 Temporal Charts**

The system shall display time-series visualizations.

### **F3.3 Statistical Indicators**

The system shall calculate averages, minima, maxima, and heat indicators.

---

## **F4 – Exporting**

### **F4.1 CSV Export**

Users shall export datasets in CSV format.

### **F4.2 PNG Export**

Users shall export charts and maps as PNG images.

---

## **F5 – Personalization**

### **F5.1 Save Preferences**

Users shall save dashboard configurations.

### **F5.2 Restore Preferences**

The system shall restore saved settings after login.

The system shall allow authenticated users to export queried meteorological observations as CSV files and download generated charts or map visualizations as PNG images for external analysis and reporting purposes.

---

## **7 Non-Functional Requirements**

| <b>ID</b> | <b>Category</b> | <b>Requirement</b>                              | <b>Priority</b> |
|-----------|-----------------|-------------------------------------------------|-----------------|
| NF-01     | Performance     | Standard queries must complete within 3 seconds | High            |
| NF-02     | Reliability     | API failures must be handled gracefully         | High            |

| ID    | Category         | Requirement                                 | Priority |
|-------|------------------|---------------------------------------------|----------|
| NF-03 | Security         | Passwords must be hashed and protected      | High     |
| NF-04 | Usability        | Dashboard must be intuitive and interactive | High     |
| NF-05 | Compatibility    | System must support major browsers          | Medium   |
| NF-06 | Maintainability  | Architecture must remain modular            | High     |
| NF-07 | Scalability      | Database must support additional cities     | Medium   |
| NF-08 | Interoperability | Backend must expose JSON REST APIs          | High     |
| NF-09 | Data Integrity   | Duplicate observations must be prevented    | High     |

---

## 8 Constraints and Dependencies

### 8.1 Technical Constraints

- Python and Flask must be used for backend development.
- PostgreSQL/PostGIS must be used for storage.
- Dashboard must use Jupyter Notebook.
- Meteostat integration requires RapidAPI authentication.

---

### 8.2 Data Constraints

- API quota limited to 500 requests/month.
- Data quality depends on Meteostat coverage.
- Some stations may contain missing values.

---

### 8.3 Legal and Organisational Constraints

- Only publicly available weather data may be used.
- Team collaboration must occur through GitHub.
- Deliverables must respect course deadlines.

---

### 8.4 External Tool Dependencies

The system depends on:

- Flask
- PostgreSQL/PostGIS
- Pandas
- GeoPandas
- Plotly
- Folium
- Jupyter Notebook
- GitHub
- Meteostat API
- Rapid API

---

## 9 Future Considerations

Potential future improvements include:

- Real-time monitoring.
- Heatwave forecasting.
- Machine-learning analytics.
- Mobile-friendly dashboards.
- Satellite imagery integration.
- Automated continuous ingestion.
- Public alert systems.

---

## 10 Appendices and References

### 10.1 External Data Sources

| ID    | Dataset                     | Source        | Purpose                        |
|-------|-----------------------------|---------------|--------------------------------|
| DS-01 | Hourly Weather Observations | Meteostat     | Meteorological analysis        |
| DS-02 | Station Metadata            | Meteostat     | Geospatial station information |
| DS-03 | Administrative Boundaries   | OpenStreetMap | Geographic visualization       |

---

## 10.2 Tools, Libraries and Frameworks

| Layer           | Technology         | Purpose                     |
|-----------------|--------------------|-----------------------------|
| Backend         | Flask              | REST API                    |
| Database        | PostgreSQL/PostGIS | Geospatial storage          |
| Data Processing | Pandas, GeoPandas  | Cleaning and processing     |
| Visualization   | Plotly, Folium     | Interactive charts and maps |
| Dashboard       | Jupyter Notebook   | Interactive analysis        |
| Version Control | GitHub             | Collaboration               |

---

## 10.3 Internal Artefacts and Diagrams

- Use Case Diagram
  - Class Diagram
  - Component Diagram
  - Database Schema
  - REST API Documentation
- 

## 10.4 Acronyms and Abbreviations

### Acronym Meaning

|       |                                   |
|-------|-----------------------------------|
| API   | Application Programming Interface |
| CSV   | Comma-Separated Values            |
| DB    | Database                          |
| GIS   | Geographic Information System     |
| JSON  | JavaScript Object Notation        |
| REST  | Representational State Transfer   |
| UI/UX | User Interface/User Experience    |
| UHI   | Urban Heat Island                 |

## **Acronym Meaning**

WSGI      Web Server Gateway Interface

---

## **Conclusion**

This document defines the requirements and analysis for the “City-Level Heat Monitoring using Meteostat” system. The proposed client-server architecture supports weather data retrieval, storage, processing, visualization, and analysis to assist users in understanding urban heat dynamics through interactive geospatial technologies.